
Data Engineering Project Report

Real-Time Car Park Analytics

Author: Damir Smetov

1. API Justification

The project utilizes the Transport for NSW Car Park API (v2.3). This data source was selected because it aligns well with the project requirements:

- **Update Frequency:** Data is updated every 1–10 minutes (triggered by counter changes at parking facilities), making it suitable for demonstrating real-time streaming via Kafka.
 - **Reliability:** The API is provided by the NSW Government, ensuring high availability and well-maintained documentation.
 - **Format:** Structured JSON responses allow efficient parsing and transformation.
 - **Analytical Value:** Provides real-world occupancy metrics (total spots vs. occupied spots), enabling meaningful urban mobility and traffic analytics.
-

2. Kafka Topic Schema

To connect the data ingestion stage (Job 1) with the processing stage (Job 2), a Kafka topic is implemented:

- **Topic Name:** `raw_events`
 - **Message Format:** JSON
 - **Key Fields:**
 - `facility_id` (Integer): Unique identifier of the parking facility
 - `tsn` (String): Transit Stop Number
 - `total` (Integer): Current number of occupied spaces
 - `spots` (Integer): Total parking capacity
 - `MessageDate` (String): Timestamp provided by the API
-

3. Data Cleaning Rules

During Job 2 (ETL process), the following data cleaning and transformation steps are applied using **Pandas**:

1. **Type Validation:** Numerical fields (`total`, `spots`) are explicitly cast to integers.
 2. **Null Handling:** Records with missing critical fields (`facility_id`, `total`) are removed.
 3. **Feature Engineering:**
A derived field `available_spots` is calculated as:
`available_spots = spots - total`
 4. **Time Normalization:**
The `MessageDate` field is converted into ISO-8601 format for consistency in storage.
-

4. SQLite Schema

Processed data is stored in an SQLite database (`app.db`) using two tables:

Table 1: `events` (Cleaned Transactional Data)

- `id`: INTEGER PRIMARY KEY
- `facility_id`: INTEGER
- `occupancy`: INTEGER
- `total_spots`: INTEGER
- `available_spots`: INTEGER
- `recorded_at`: DATETIME

Table 2: `daily_summary` (Aggregated Analytical Data)

- `id`: INTEGER PRIMARY KEY
 - `date`: DATE
 - `facility_id`: INTEGER
 - `avg_occupancy`: FLOAT
 - `peak_occupancy`: INTEGER
 - `peak_hour`: INTEGER
-

5. Implementation Evidence

5.1 Airflow DAGs Overview

Confirms that ingestion, cleaning, and analytics DAGs are active and scheduled.

The screenshot shows the Apache Airflow web interface with the 'Dags' tab selected. The interface displays a list of DAGs with their respective schedules, latest run status, and next run time. The DAGs are: job1_ingestion_dag (Schedule: */1 * * * *, Latest Run: 2025-12-19 03:41:25, Next Run: 2025-12-19 03:42:00), job2_clean_store_dag (Schedule: 0 * * * *, Latest Run: 2025-12-19 03:25:36, Next Run: 2025-12-19 04:00:00), and job3_daily_summary_dag (Schedule: 0 0 * * * *, Latest Run: 2025-12-19 03:30:57, Next Run: 2025-12-19 05:00:00). The status of each DAG is indicated by a colored circle: green for Success, blue for Running, and red for Failed. A bar chart on the right shows the execution history of the DAGs.

5.2 Job 1: Ingestion (API → Kafka)

Logs confirm successful data ingestion from the API into Kafka topics.

The screenshot shows the Apache Airflow web interface with the 'Dag Run' tab selected for the DAG 'job1_ingestion_dag'. The task 'produce_data' is highlighted, showing its status as 'Success'. The log viewer on the right displays the output of the task, confirming successful data ingestion from the API into Kafka topics. The logs show the following information: Operator: PythonOperator, Start Date: 2025-12-19 03:15:20, End Date: 2025-12-19 03:15:33, Duration: 00:00:26.007. The log viewer also shows the task's execution details, including the source of the data and the Kafka topic.

5.3 Job 2: Cleaning & Storage (Kafka → SQLite)

Consumer logs show successful transformation and insertion into the database.

The screenshot displays the Apache Airflow web interface. On the left, a sidebar contains navigation links for Home, Dags, Assets, Browse, Admin, Docs, and User. The main canvas shows a DAG named 'job2_clean_store_dag' with a task named 'clean_data' highlighted by a blue box. The task is a 'PythonOperator' and its status is 'success'. On the right, a detailed view of the 'clean_data' task is shown, including its execution details: Operator: PythonOperator, Start Date: 2025-12-19 14:41:18, End Date: 2025-12-19 14:41:26, Duration: 00:00:16.532, and Dag Version: v116. Below this, a log viewer shows various INFO messages, such as 'Updated partition assignment', 'Setting newly assigned partitions', and 'Inserting records source=task.stdout'.

5.4 Job 3: Daily Analytics

Aggregation logic successfully computes daily metrics (e.g., for 2025-12-18).

The screenshot displays the Apache Airflow web interface for a different DAG. The main canvas shows a DAG named 'job3_daily_summary_dag' with a task named 'run_daily_analytics' highlighted by a blue box. The task is a 'PythonOperator' and its status is 'success'. On the right, a detailed view of the 'run_daily_analytics' task is shown, including its execution details: Operator: PythonOperator, Start Date: 2025-12-19 03:30:58, End Date: 2025-12-19 03:30:58, Duration: 00:00:00.310, and Dag Version: v1. Below this, a log viewer shows various INFO messages, such as 'DAG bundles loaded', 'Filling up the DagBag', and 'Daily analytics computed for 2025-12-18'.

6. Conclusion

The implemented pipeline successfully combines **real-time streaming (Kafka)** with **batch orchestration (Airflow)**.

All components operate cohesively within a containerized environment, meeting the technical requirements for high-frequency ingestion, automated ETL, and analytical reporting.